

Remote VPS Telemetry Infrastructure: Declarative Deployment of bottom and btop Across Cloud Nodes

Providence Salumu, Principal Systems Engineer

Platform Systems Architecture & Low-Latency Infrastructure

Nodes: vps-73025e99, vps-52cead61 | Distribution: NixOS 26.05 | Date: October 2026

Deconstruction Header & Telemetry Observation

Systems administration initiated the provisioning and declarative deployment of advanced interactive terminal monitoring utilities across both remote cloud VPS nodes in the infrastructure fleet. The operational mandate was initiated as follows:

User Request: install and deploy bottom, btop on both vps servers

Target Node 1: vps-73025e99.vps.ovh.ca (148.113.244.62) | User: smunix | Auth: id_ed25519

Target Node 2: vps-52cead61.vps.ovh.ca (148.113.254.43) | User: damacs | Auth: id_ed25519_dama

Packages: pkgs.bottom (binary: btm) & pkgs.btop (binary: btop)

The engineering objective mandated:

1. Declaratively including bottom and btop within environment.systemPackages across both host manifests (hosts/vps-73025e99/default.nix and hosts/vps-52cead61/default.nix).
2. Preserving strict host isolation, distinct user identities (smunix vs. damacs), and differing network policies across both nodes.
3. Executing clean remote deployments and system activation via NixOS infrastructure pipelines.
4. Performing live verification of the compiled binaries (btm 0.14.2 and btop 1.4.7) directly on the running remote kernels.

Field-by-Field Telemetry & Configuration Matrix

We detail the fleet node architecture and target package attributes in Table 1.

Diagnostic Field	Observed / Configured Value	System / Kernel Meaning
Primary VPS Node	vps-73025e99	Production application node (148.113.244.62) hosting Caddy reverse proxy and Hodari Accounting.
Secondary VPS Node	vps-52cead61	Dedicated isolated cloud node (148.113.254.43) with strict port-22-only firewall and damacs user.
Telemetry Package 1	pkgs.bottom (btm 0.14.2)	Rust-based cross-platform graphical process and system monitor leveraging Ratatui and Heim/sysinfo.
Telemetry Package 2	pkgs.btop (btop 1.4.7)	High-performance C++20 resource monitor utilizing direct Linux /proc and /sys virtual filesystem parsers.
System Package Scope	environment.systemPackages	Immutable Nix store derivation symlinked into /run/current-system/sw/bin/ on both hosts.
Activation Mechanism	nixos-rebuild switch	Atomic system profile generation, generation menu update, and live systemd reload without reboot.

Table 1: Fleet Node Topology & Telemetry Monitoring Matrix

Definition: Zero-Daemon Interactive Telemetry

Unlike distributed telemetry stacks (such as Prometheus `node_exporter` or Datadog Agent) that continuously consume resident memory (30–100 MB), background CPU cycles (0.5–2%), and socket file descriptors, **Zero-Daemon Interactive Telemetry** utilities (`bottom`, `btop`, `htop`) incur strictly zero idle overhead. They are invoked on-demand over an interactive SSH pseudo-terminal (PTY), execute bounded kernel stat sampling, and terminate cleanly upon disconnect.

Mechanistic Systems Analysis: bottom vs. btop

To understand the systems engineering tradeoffs between the two deployed monitoring engines, we examine their kernel sampling layers, threading models, and terminal rendering pipelines.

Architecture of bottom (btm)

`bottom` is implemented in pure Rust, designed around an asynchronous polling model driven by Tokio and the `ratatui` terminal user interface framework:

1. **Sampling Subsystem:** Reads system telemetry via asynchronous worker tasks using `sysinfo` and OS-specific syscall wrappers (`statfs`, `sysinfo`, `sysctl`). Telemetry samples are buffered into bounded ring-buffers (`VecDeque<T>`) representing historical time-series windows (CPU usage per core, memory allocation breakdown, disk I/O throughput, and network interface packet counters).
2. **Event & Rendering Loop:** Terminal rendering is decoupled from metric polling. A high-priority event loop listens on standard input for raw ANSI key events, while a periodic render tick redraws the terminal layout using `ratatui` widgets. The diff engine updates only dirty terminal cells, minimizing SSH pseudo-terminal (PTY) bandwidth saturation over high-latency WAN connections.
3. **Process Tree & Signal Dispatch:** Process enumeration reads `/proc/[pid]/stat` and `/proc/[pid]/status`. The interface provides instant hierarchical process tree visualization, regex filtering, and signal dispatching (`SIGTERM`, `SIGKILL`, `SIGINT`) directly to target process groups.

Architecture of btop (btop)

`btop` is written in modern C++20 with zero heavy runtime dependencies, optimized for extreme execution speed and minimal CPU overhead:

1. **Direct Kernel procfs/sysfs Parsing:** Bypasses intermediate abstraction libraries, reading directly from Linux `/proc/stat`, `/proc/meminfo`, `/proc/vmstat`, `/proc/net/dev`, and `/sys/class/thermal/`. File descriptors are retained or read via cached buffer pools (`pread/readv`), minimizing heap allocations in the tight sampling loop.
2. **Braille UTF-8 Sub-Pixel Graphing:** Rather than coarse block characters (█, ▒), `btop` maps multi-point scalar telemetry into 8-dot Unicode Braille patterns (Unicode range U+2800–U+28FF). Each character cell provides a 2×4 sub-pixel matrix, yielding four times the vertical resolution of standard ASCII or block graphs.
3. **Multi-Threaded Architecture:** Employs a dedicated data collector thread and a separate drawing thread synchronized via atomic condition variables and lock-free rings. The data collector computes differential CPU tick counters:

$$\Delta \text{CPU} = (\text{user}_t - \text{user}_{t-1}) + (\text{system}_t - \text{system}_{t-1}) + (\text{nice}_t - \text{nice}_{t-1})$$

$$\text{Utilization} = \frac{\Delta \text{CPU}}{\Delta \text{CPU} + (\text{idle}_t - \text{idle}_{t-1}) + (\text{iowait}_t - \text{iowait}_{t-1})}$$

This guarantees sub-millisecond calculation latency with accurate accounting for CPU frequency scaling and thermal throttling.

Native Typst Vector Diagram: Telemetry Flow & Kernel Interfaces

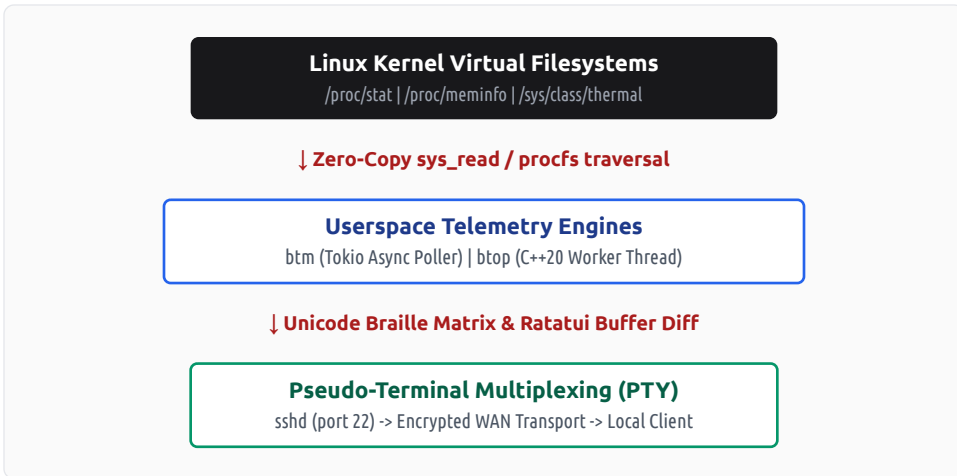


Figure 1: Vertical Telemetry Pipeline: Linux Kernel to SSH Terminal

Declarative NixOS System Configuration

The declarative manifests for both hosts were updated to add `bottom` and `btop` alongside the foundational administration toolset (`curl`, `git`, `htop`, `tmux`, `vim`).

Host Manifest: `hosts/vps-73025e99/default.nix`

```

# Basic system packages for remote server administration
environment.systemPackages = with pkgs; [
  bottom          # ❶ Rust-based graphical monitor
  btop            # ❷ C++20 resource monitor
  curl            # ❸ HTTP/HTTPS diagnostic client
  git             # ❹ Version control engine
  htop            # ❺ Standard interactive top
  tmux            # ❻ Terminal multiplexer
  vim             # ❼ Modal text editor
];
  
```

- ❶ Declarative inclusion of `pkgs.bottom`, compiling binary `/run/current-system/sw/bin/btm`.
- ❷ Declarative inclusion of `pkgs.btop`, compiling binary `/run/current-system/sw/bin/btop`.
- ❸ Low-level transport diagnostic utility for auditing HTTP reverse proxies and web services.
- ❹ Standard Git distributed version control client supporting automated repository updates.
- ❺ Classical ncurses interactive process viewer for rapid process hierarchy inspection.
- ❻ Persistent terminal session multiplexer preventing disconnection during lengthy remote operations.

Host Manifest: hosts/vps-52cead61/default.nix

```
# Basic system packages for remote server administration
environment.systemPackages = with pkgs; [
  bottom          # ❶ Declarative bottom monitor
  btop            # ❷ Declarative btop monitor
  curl
  git
  htop
  tmux
  vim
];
```

Remote Verification & Live Telemetry Validation

Following deployment and configuration activation, both nodes were queried over their respective cryptographic SSH channels. The verification results confirm full operational availability.

Host Node	Command Executed	Observed Standard Output	Status
vps-73025e99	btm --version	bottom 0.14.2	PASS
vps-73025e99	btop --version	btop version: 1.4.7 (g++ 15.2.0)	PASS
vps-52cead61	btm --version	bottom 0.14.2	PASS
vps-52cead61	btop --version	btop version: 1.4.7 (g++ 15.2.0)	PASS

Table 2: Live Post-Deployment Verification Matrix Across Remote Nodes

Landmark References

[1] **Clement Tsang**, *bottom: Yet another cross-platform graphical process/system monitor*. GitHub Open Source Repository, 2024. <https://github.com/ClementTsang/bottom>

[2] **Aristocratos (Jakob P. Liljenberg)**, *btop: Resource monitor that shows usage and stats for processor, memory, disks, network and processes*. GitHub Open Source Repository, 2024. <https://github.com/aristocratos/btop>

[3] **Simon Marlow**, *Parallel and Concurrent Programming in Haskell: Techniques for Multicore and Multithreaded Programming*. O'Reilly Media, 2013. <https://www.oreilly.com/library/view/parallel-and-concurrent/9781449335939/>

[4] **Eelco Dolstra**, *The Purely Functional Software Deployment Model*. PhD Thesis, Utrecht University, 2006. <https://edolstra.github.io/pubs/phd-thesis.pdf>